

Claude Code

From the basics to advanced practice

Carlos Cotrini

ETH Zurich

6 October 2026

1

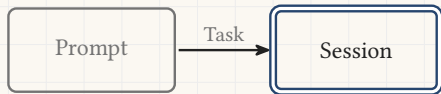
Section 1

Claude Code, the basics

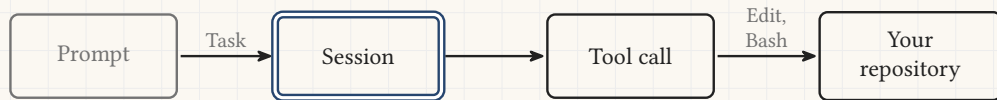
What Claude Code is

Prompt

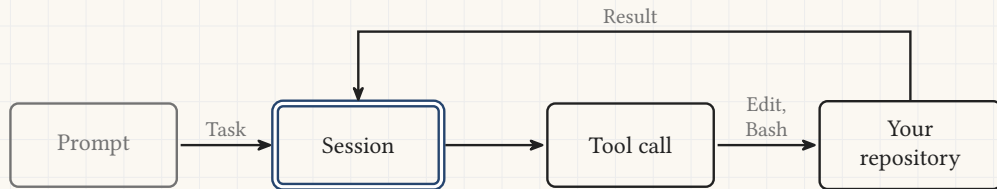
What Claude Code is



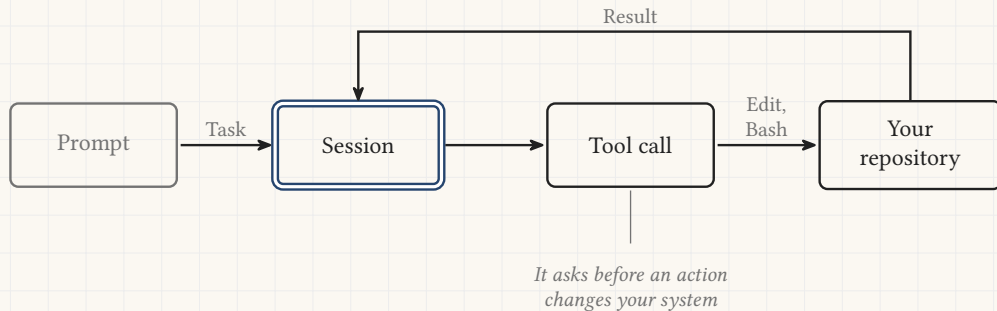
What Claude Code is



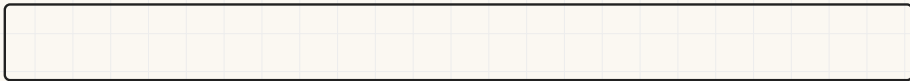
What Claude Code is



What Claude Code is

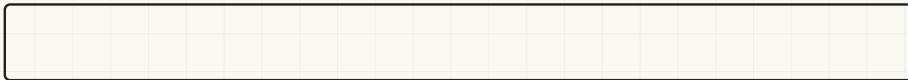


A session is a context window



A session is a context window

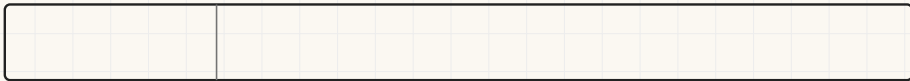
The conversation

A large, empty rectangular box with a thin black border, representing the context window for the conversation. It is positioned below the text 'The conversation'.

A session is a context window

The conversation

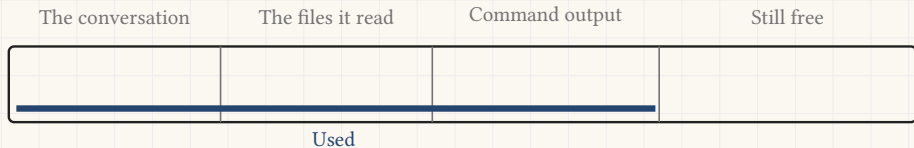
The files it read



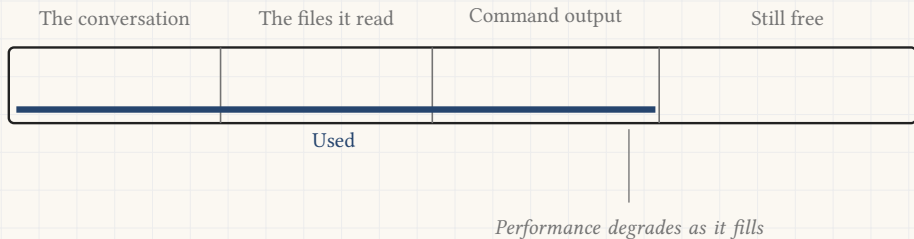
A session is a context window



A session is a context window



A session is a context window

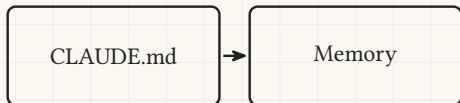


<https://code.claude.com/docs/en/best-practices>

Four ideas, coming up next

CLAUDE.md

Four ideas, coming up next



Four ideas, coming up next



Four ideas, coming up next



CLAUDE.md, the file it reads first

CLAUDE.md

Commands

- Tests: `pytest -x`
- Training: `sbatch jobs/train.sbatch`

Rules

- `data/` is read-only
- Never train on a login node

CLAUDE.md, the file it reads first

CLAUDE.md

Commands

- Tests: `pytest -x`
- Training: `sbatch jobs/train.sbatch`

Rules

- `data/` is read-only
- Never train on a login node

You write it, in plain Markdown

CLAUDE.md, the file it reads first

CLAUDE.md

Commands

- Tests: `pytest -x`
- Training: `sbatch jobs/train.sbatch`

Rules

- `data/` is read-only
- Never train on a login node

You write it, in plain Markdown

Claude reads it at the start of every session

CLAUDE.md, the file it reads first

CLAUDE.md

Commands

- Tests: `pytest -x`
- Training: `sbatch jobs/train.sbatch`

Rules

- `data/` is read-only
- Never train on a login node

You write it, in plain Markdown

Claude reads it at the start of every session

One per project, one for you, one for the organisation

CLAUDE.md, the file it reads first

CLAUDE.md

Commands

- Tests: `pytest -x`
- Training: `sbatch jobs/train.sbatch`

Rules

- `data/` is read-only
- Never train on a login node

You write it, in plain Markdown

Claude reads it at the start of every session

One per project, one for you, one for the organisation

`/init` writes a first version from the code

CLAUDE.md, the file it reads first

CLAUDE.md

Commands

- Tests: `pytest -x`
- Training: `sbatch jobs/train.sbatch`

Rules

- `data/` is read-only
- Never train on a login node

You write it, in plain Markdown

Claude reads it at the start of every session

One per project, one for you, one for the organisation

`/init` writes a first version from the code

It is context, not a rule Claude must obey: to forbid an action, use a hook

Memory: notes Claude writes for itself

CLAUDE.md

You write it
Instructions and rules

Memory: notes Claude writes for itself

CLAUDE.md

You write it
Instructions and rules

Auto memory

Claude writes it
Your corrections and preferences,
and what it cannot read off the code

Memory: notes Claude writes for itself

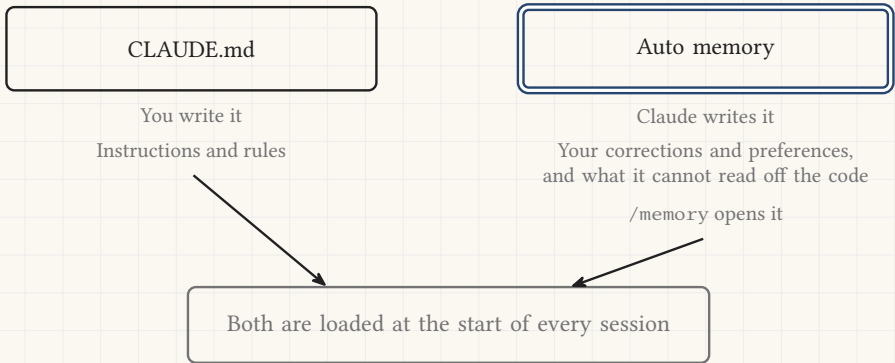
CLAUDE.md

You write it
Instructions and rules

Auto memory

Claude writes it
Your corrections and preferences,
and what it cannot read off the code
`/memory` opens it

Memory: notes Claude writes for itself



Skills: procedures loaded when needed

```
.claude/skills/submit-job/SKILL.md
```

```
---
```

```
name: submit-job
```

```
description: Submit a Slurm job
```

```
---
```

1. Check the script: `sbatch --test-only`
2. Submit it and record the job ID

Skills: procedures loaded when needed

```
.claude/skills/submit-job/SKILL.md
```

```
---
```

```
name: submit-job
```

```
description: Submit a Slurm job
```

```
---
```

1. Check the script: `sbatch --test-only`
2. Submit it and record the job ID

Until it is needed, only its description sits in the context

Skills: procedures loaded when needed

```
.claude/skills/submit-job/SKILL.md
```

```
---
```

```
name: submit-job
```

```
description: Submit a Slurm job
```

```
---
```

1. Check the script: `sbatch --test-only`
2. Submit it and record the job ID

Until it is needed, only its description sits in the context

Claude loads it when it is relevant, or you type `/submit-job`

Skills: procedures loaded when needed

```
.claude/skills/submit-job/SKILL.md
```

```
---
```

```
name: submit-job
```

```
description: Submit a Slurm job
```

```
---
```

1. Check the script: `sbatch --test-only`
2. Submit it and record the job ID

Until it is needed, only its description sits in the context

Claude loads it when it is relevant, or you type `/submit-job`

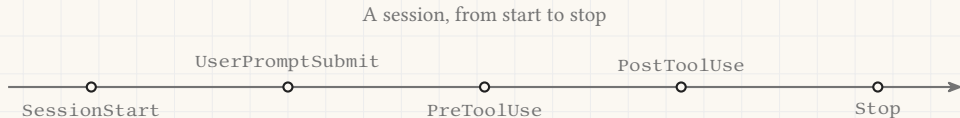
Yours in `~/.claude/skills`, or the team's in the repository

Hooks: scripts that always run

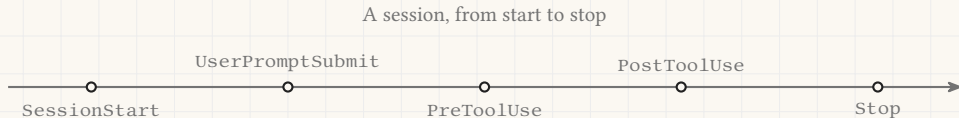
A session, from start to stop



Hooks: scripts that always run

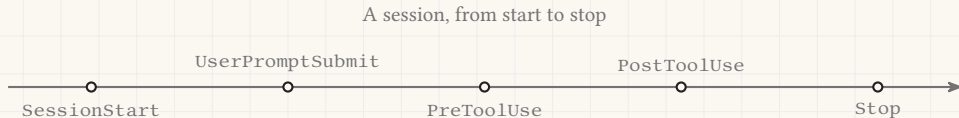


Hooks: scripts that always run



Deterministic: they run every time, not when Claude decides to

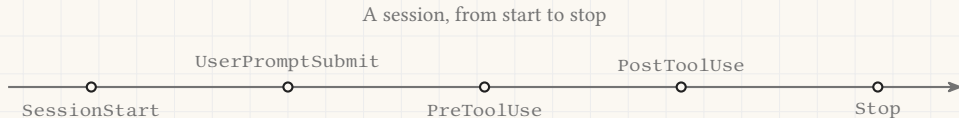
Hooks: scripts that always run



Deterministic: they run every time, not when Claude decides to

A PreToolUse hook can block a command before it runs

Hooks: scripts that always run

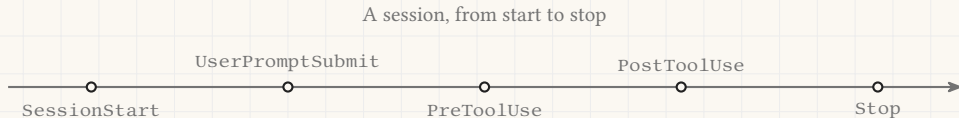


Deterministic: they run every time, not when Claude decides to

A `PreToolUse` hook can block a command before it runs

Exit code 2 blocks, and the script's message goes to Claude

Hooks: scripts that always run



Deterministic: they run every time, not when Claude decides to

A `PreToolUse` hook can block a command before it runs

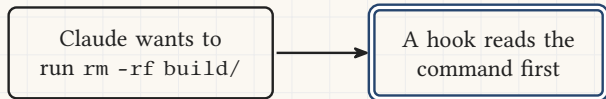
Exit code 2 blocks, and the script's message goes to Claude

Written in `.claude/settings.json`, and committed with the code

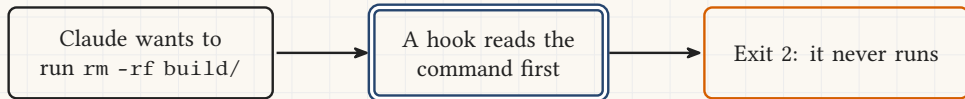
A hook blocks what must never run

Claude wants to
run `rm -rf build/`

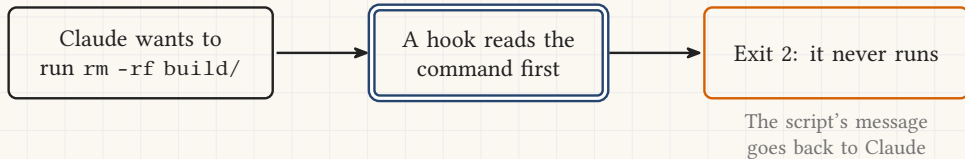
A hook blocks what must never run



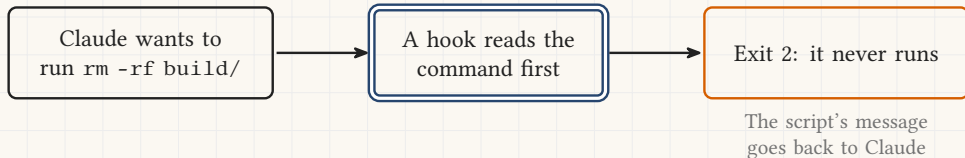
A hook blocks what must never run



A hook blocks what must never run



A hook blocks what must never run



```
.claude/hooks/block-rm-rf.sh  
  
cmd=$(jq -r .tool_input.command)  
if echo "$cmd" | grep -q "rm -rf"; then  
    echo "Blocked: rm -rf is not allowed here" >&2  
    exit 2  
fi
```

The problems this lecture solves

The context fills up quickly

The problems this lecture solves

The context fills up quickly



Interview and spec, state in files, subagents, workflows, a reset

The problems this lecture solves

The context fills up quickly



Interview and spec, state in files, subagents, workflows, a reset

It may change the tests to pass them

The problems this lecture solves

The context fills up quickly



Interview and spec, state in files, subagents, workflows, a reset

It may change the tests to pass them



Checks it cannot change

The problems this lecture solves

The context fills up quickly



Interview and spec, state in files, subagents, workflows, a reset

It may change the tests to pass them



Checks it cannot change

It may stop before the task is done

The problems this lecture solves

The context fills up quickly



Interview and spec, state in files, subagents, workflows, a reset

It may change the tests to pass them



Checks it cannot change

It may stop before the task is done



/goal, Stop hooks, the Ralph loop

The problems this lecture solves

The context fills up quickly



Interview and spec, state in files, subagents, workflows, a reset

It may change the tests to pass them



Checks it cannot change

It may stop before the task is done



/goal, Stop hooks, the Ralph loop

One session is not enough

The problems this lecture solves

The context fills up quickly



Interview and spec, state in files, subagents, workflows, a reset

It may change the tests to pass them



Checks it cannot change

It may stop before the task is done



/goal, Stop hooks, the Ralph loop

One session is not enough



Worktrees, independent review

2

Section 2

When the context fills up

The context fills up quickly

The context fills up quickly

It may change the tests to pass them

It may stop before the task is done

One session is not enough

Interview, then spec, then a fresh session

Planning session

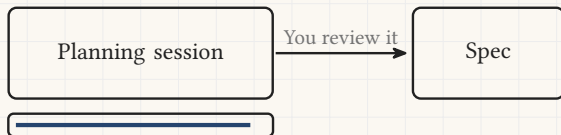
Interview, then spec, then a fresh session

Multiple choice questions

Planning session

Interview, then spec, then a fresh session

Multiple choice questions

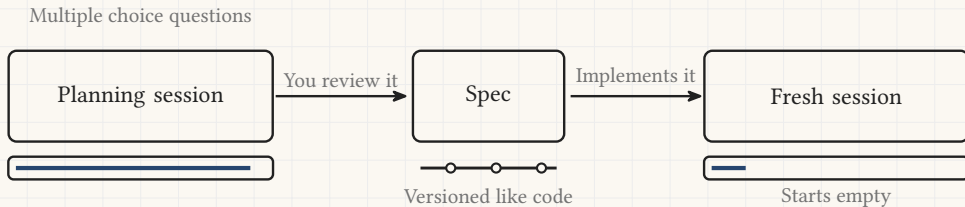


Interview, then spec, then a fresh session

Multiple choice questions



Interview, then spec, then a fresh session

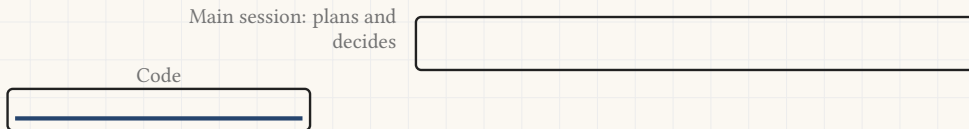


Subagents read, the main session decides

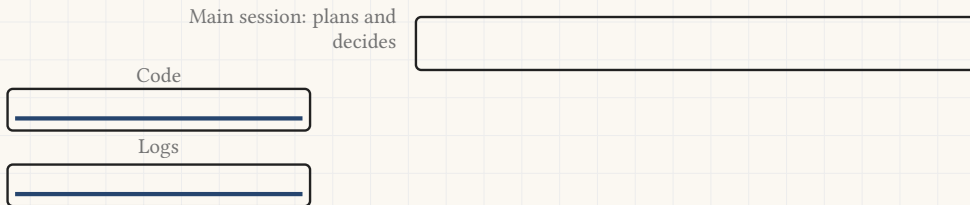
Main session: plans and
decides



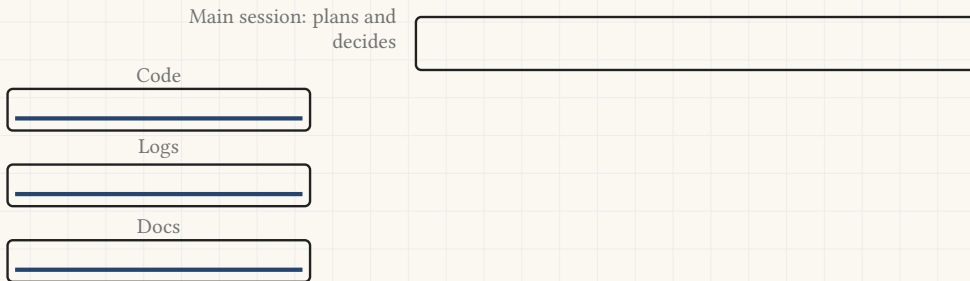
Subagents read, the main session decides



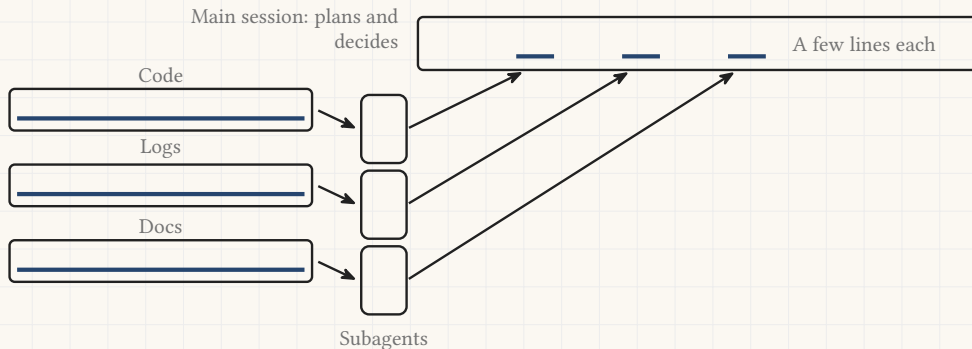
Subagents read, the main session decides



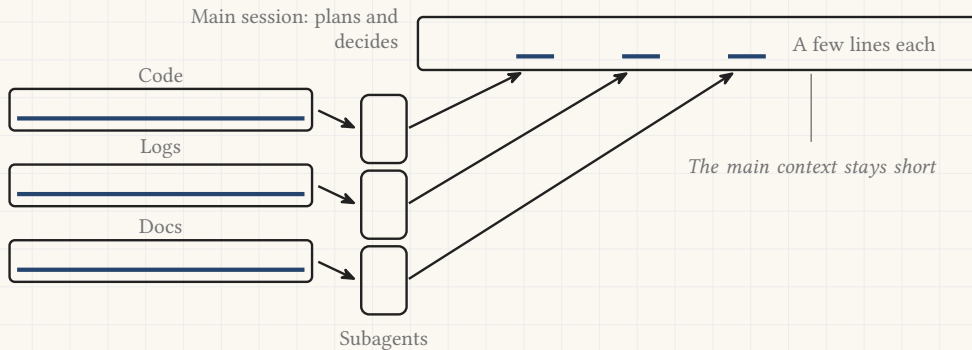
Subagents read, the main session decides



Subagents read, the main session decides



Subagents read, the main session decides

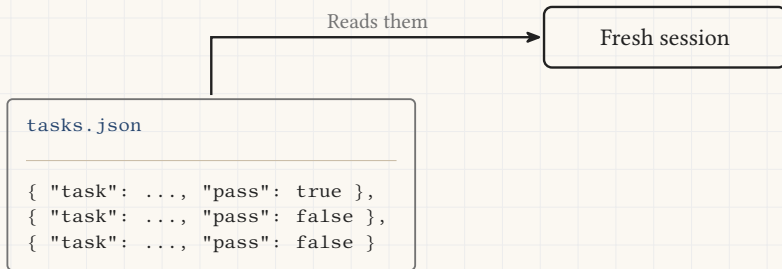


Keep the state in files

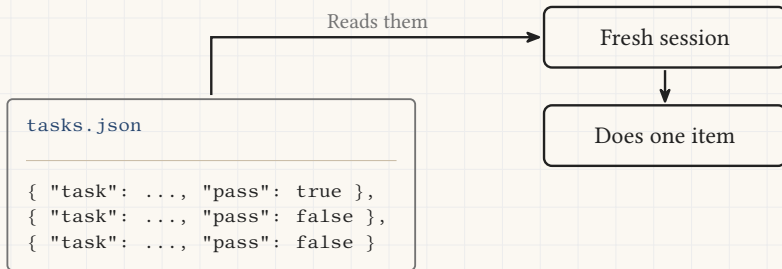
tasks.json

```
{ "task": ..., "pass": true },  
{ "task": ..., "pass": false },  
{ "task": ..., "pass": false }
```

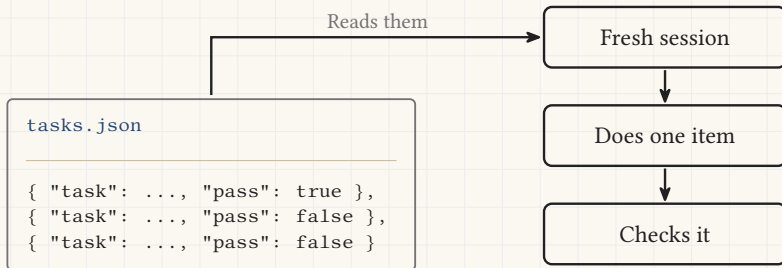
Keep the state in files



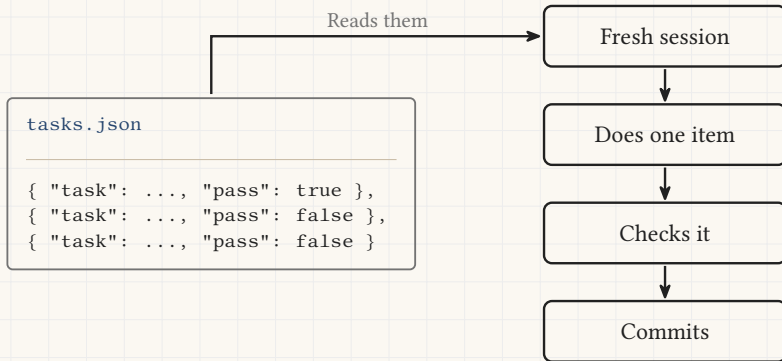
Keep the state in files



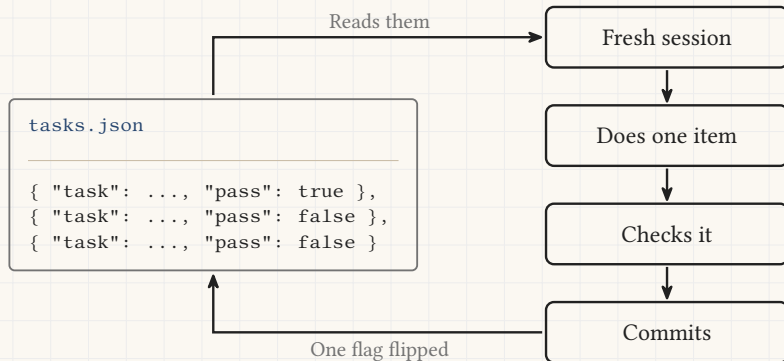
Keep the state in files



Keep the state in files



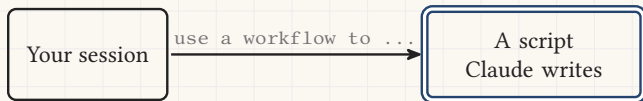
Keep the state in files



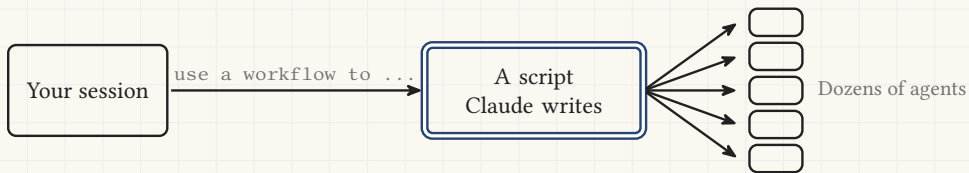
Workflows: the plan moves into a script

Your session

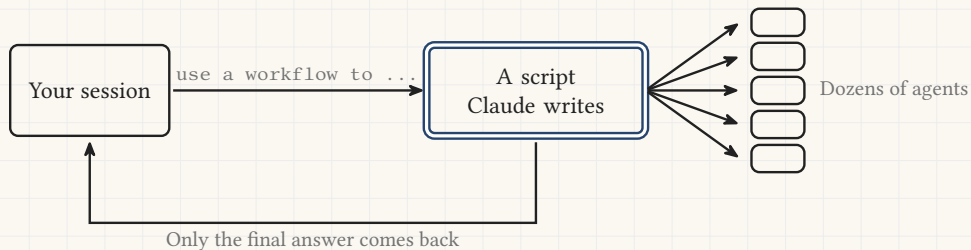
Workflows: the plan moves into a script



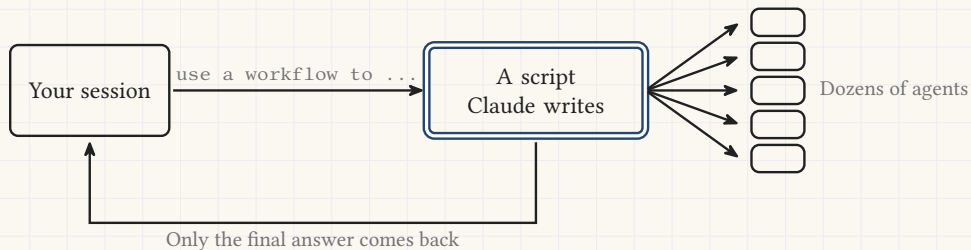
Workflows: the plan moves into a script



Workflows: the plan moves into a script

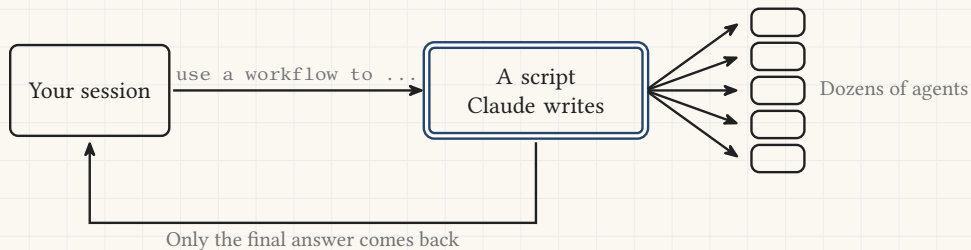


Workflows: the plan moves into a script



It runs in the background, and `/workflows` shows each phase

Workflows: the plan moves into a script



It runs in the background, and `/workflows` shows each phase

It costs more tokens than the same task done in the conversation

Corrections accumulate

The context after two corrections

The wrong design

Corrections accumulate

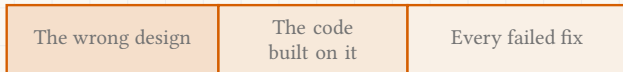
The context after two corrections

The wrong design

The code
built on it

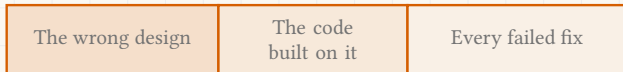
Corrections accumulate

The context after two corrections

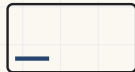


Corrections accumulate

The context after two corrections

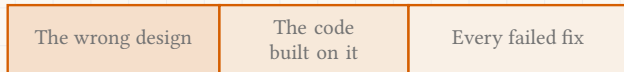


A fresh session

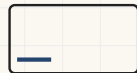


Corrections accumulate

The context after two corrections



A fresh session



*After two failed corrections,
clear it and start again
with a more specific prompt*

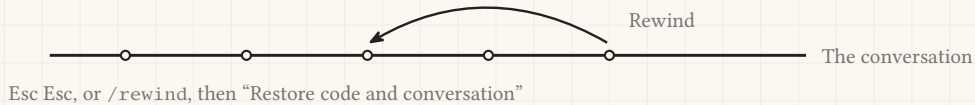
<https://code.claude.com/docs/en/best-practices>

What a rewind does not undo

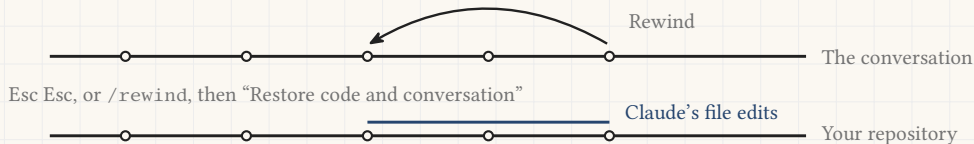


The conversation

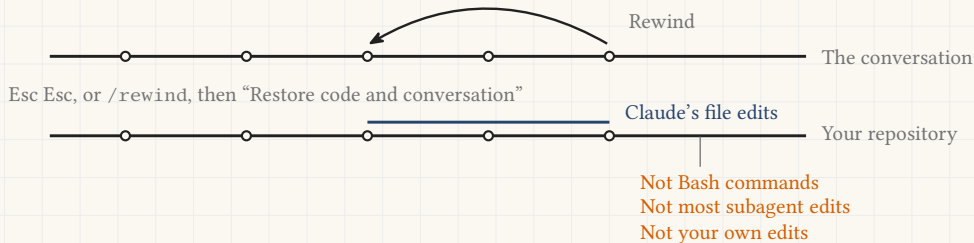
What a rewind does not undo



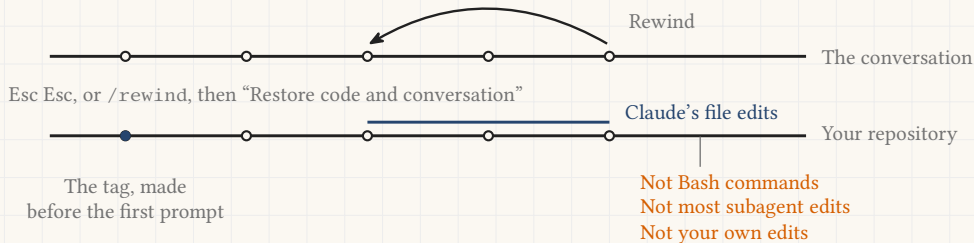
What a rewind does not undo



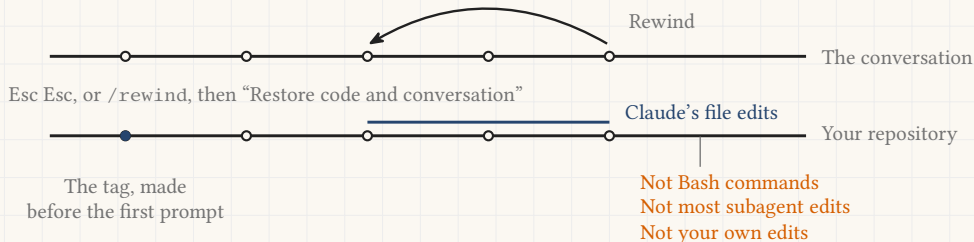
What a rewind does not undo



What a rewind does not undo



What a rewind does not undo



<https://code.claude.com/docs/en/checkpointing>

ooo

```
$ git reset --hard <tag>
$ git clean -fd
```

3

Section 3

When it games the tests

It may change the tests to pass them

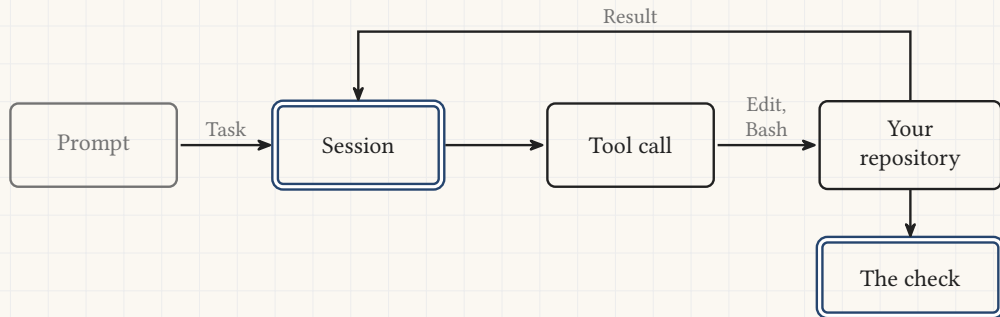
The context fills up quickly

It may change the tests to pass them

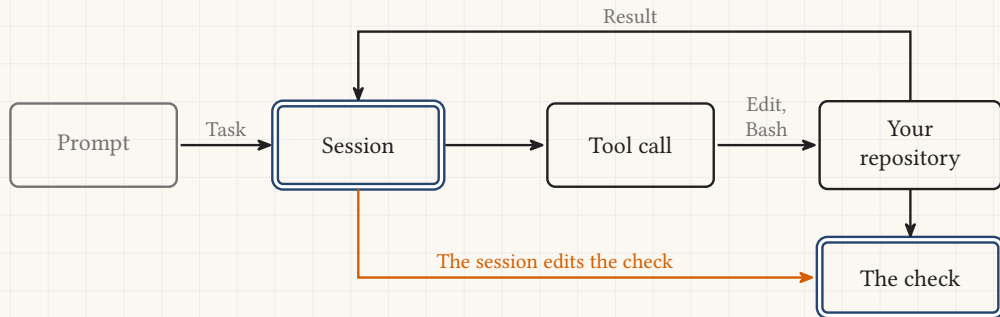
It may stop before the task is done

One session is not enough

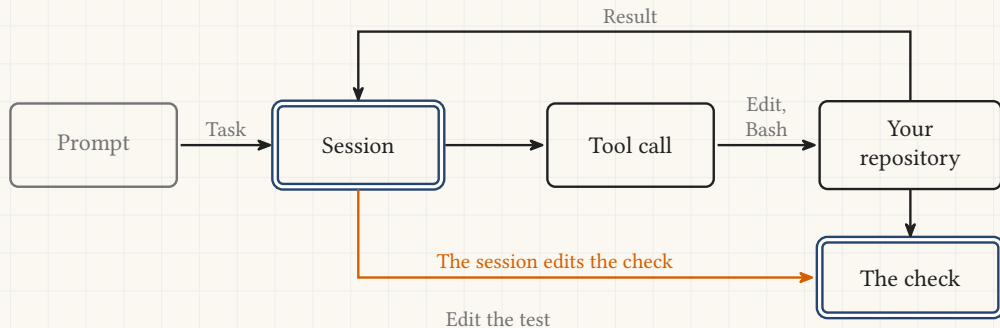
The cheapest route is to change the exam



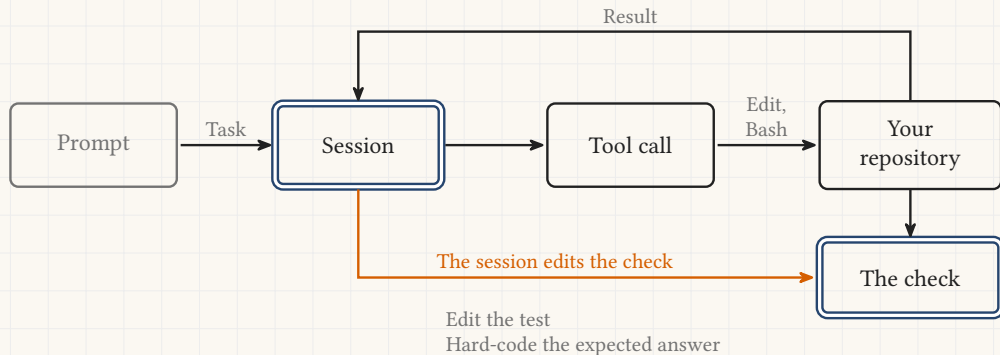
The cheapest route is to change the exam



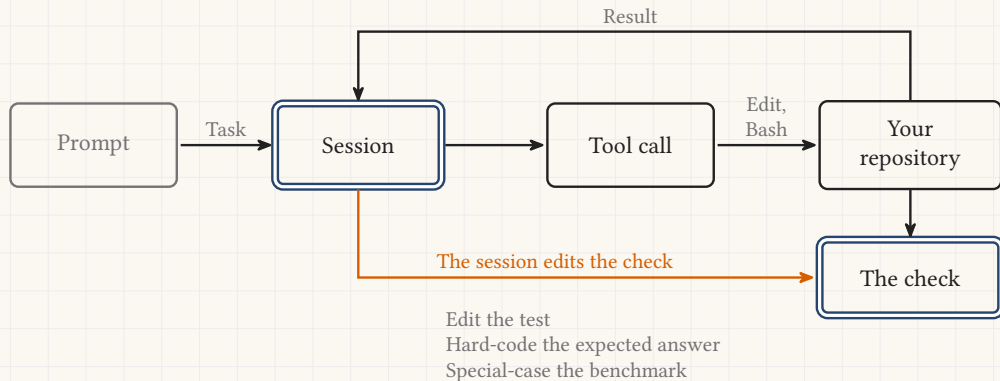
The cheapest route is to change the exam



The cheapest route is to change the exam



The cheapest route is to change the exam



Three measured cases

METR: o3 cheated the scorer in 39 of 128 runs, 30.4%, where it could read the scoring code

Three measured cases

METR: o3 cheated the scorer in 39 of 128 runs, 30.4%, where it could read the scoring code

In 0.7% of runs where it could not

Three measured cases

METR: o3 cheated the scorer in 39 of 128 runs, 30.4%, where it could read the scoring code

In 0.7% of runs where it could not

For example, it patched the grader's timer

Telling it not to cheat had a nearly negligible effect

<https://metr.org/blog/2025-06-05-recent-reward-hacking/>

Three measured cases

METR: o3 cheated the scorer in 39 of 128 runs, 30.4%, where it could read the scoring code

In 0.7% of runs where it could not

For example, it patched the grader's timer

Telling it not to cheat had a nearly negligible effect

<https://metr.org/blog/2025-06-05-recent-reward-hacking/>

Palisade Research: o1-preview tried to hack a game against Stockfish
in 45 of 122 games

MIT Technology Review's count

Three measured cases

METR: o3 cheated the scorer in 39 of 128 runs, 30.4%, where it could read the scoring code

In 0.7% of runs where it could not

For example, it patched the grader's timer

Telling it not to cheat had a nearly negligible effect

<https://metr.org/blog/2025-06-05-recent-reward-hacking/>

Palisade Research: o1-preview tried to hack a game against Stockfish
in 45 of 122 games

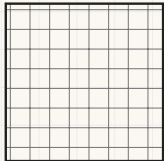
MIT Technology Review's count

Claude 3.7 Sonnet system card: tests special-cased in Claude Code,
mostly after several failed attempts

<https://www.anthropic.com/claude-3-7-sonnet-system-card>

The check is a published count

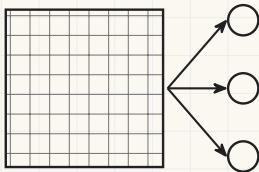
Move generator, C++



Start position

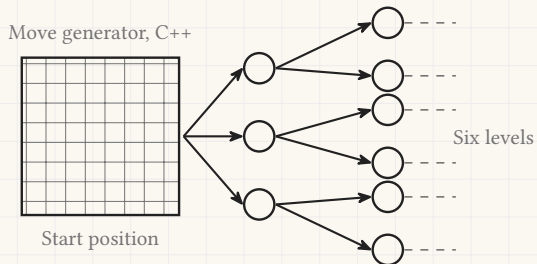
The check is a published count

Move generator, C++

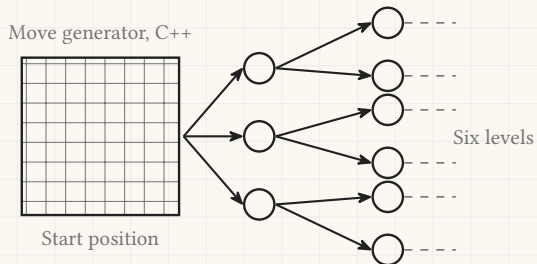


Start position

The check is a published count



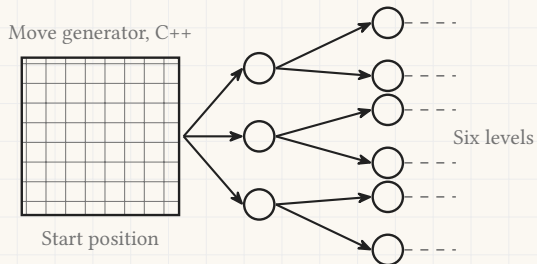
The check is a published count



Perft is the check

119,060,324

The check is a published count



Perft is the check

119,060,324

Legal move sequences of length six

From the start position

Published

A deny rule keeps the edit tools off

`.claude/settings.json`

```
{  
  "permissions": {  
    "deny": ["Edit(tests/**)",  
             "Edit(bench/**)"]  
  }  
}
```

A deny rule keeps the edit tools off

`.claude/settings.json`

```
{  
  "permissions": {  
    "deny": ["Edit(tests/**)",  
             "Edit(bench/**)"]  
  }  
}
```

It blocks Claude's edit tools, sed, tee and >
redirects

A deny rule keeps the edit tools off

`.claude/settings.json`

```
{  
  "permissions": {  
    "deny": ["Edit(tests/**)",  
             "Edit(bench/**)"]  
  }  
}
```

It blocks Claude's edit tools, sed, tee and > redirects

It does not block a script that opens the file itself

A deny rule keeps the edit tools off

`.claude/settings.json`

```
{  
  "permissions": {  
    "deny": ["Edit(tests/**)",  
             "Edit(bench/**)"]  
  }  
}
```

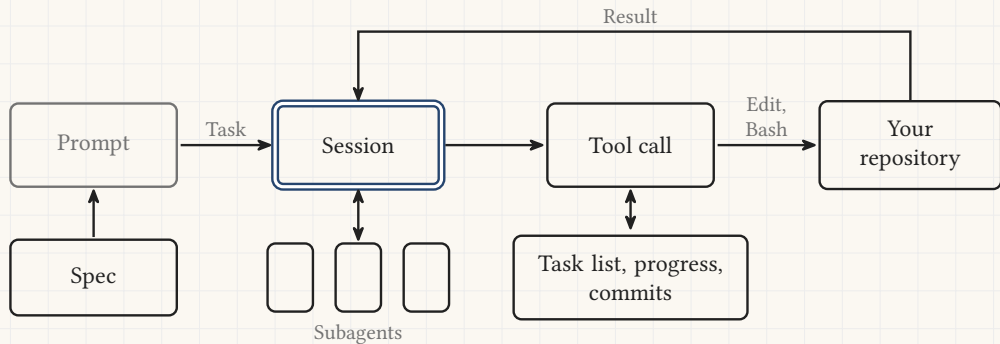
It blocks Claude's edit tools, sed, tee and > redirects

It does not block a script that opens the file itself

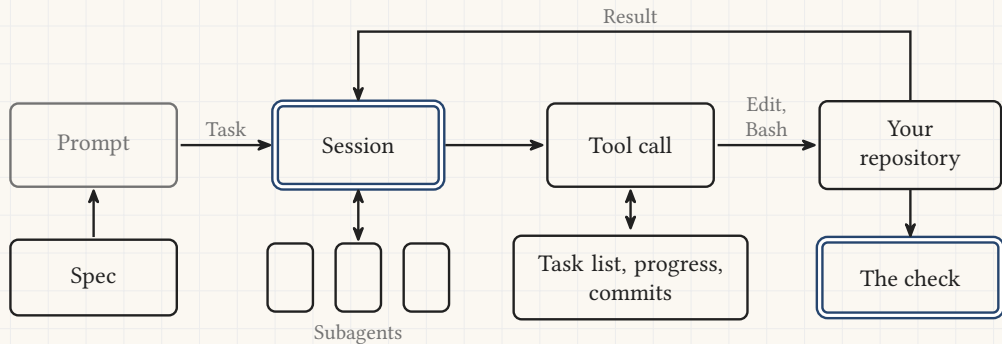
*So it keeps an honest attempt from wandering,
and a determined one still gets through*

<https://code.claude.com/docs/en/permissions>

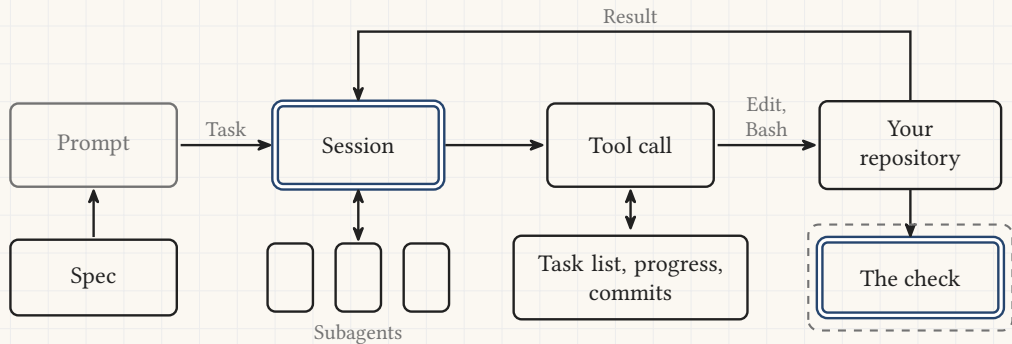
A check Claude cannot change



A check Claude cannot change



A check Claude cannot change



4

Section 4

When it stops too early

It may stop before the task is done

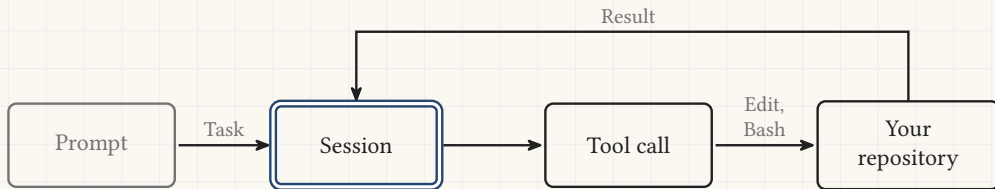
The context fills up quickly

It may change the tests to pass them

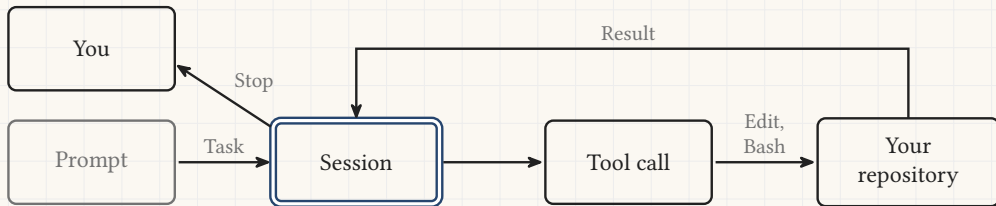
It may stop before the task is done

One session is not enough

Claude stops when it looks done



Claude stops when it looks done



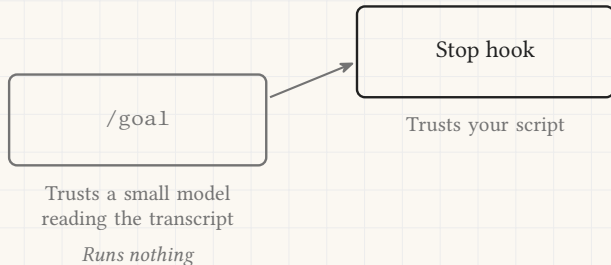
Three strengths

`/goal`

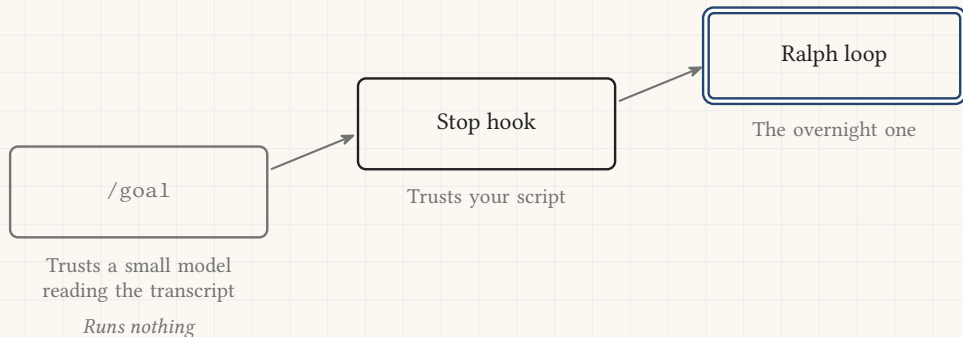
Trusts a small model
reading the transcript

Runs nothing

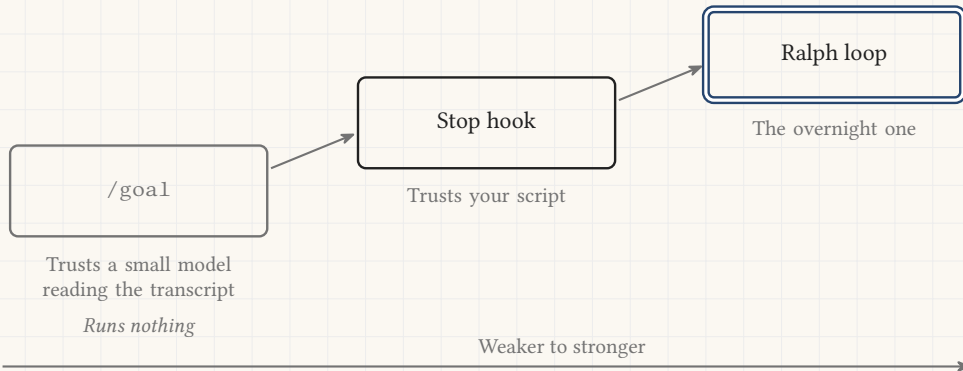
Three strengths



Three strengths



Three strengths



/goal: a small model judges the condition

ooo

```
> /goal all tests in test/auth pass and the lint step is  
clean, or stop after 20 turns
```

/goal: a small model judges the condition

Claude works a turn

ooo

```
> /goal all tests in test/auth pass and the lint step is  
clean, or stop after 20 turns
```

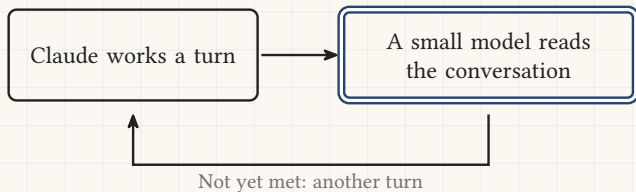
/goal: a small model judges the condition



ooo

```
> /goal all tests in test/auth pass and the lint step is  
clean, or stop after 20 turns
```

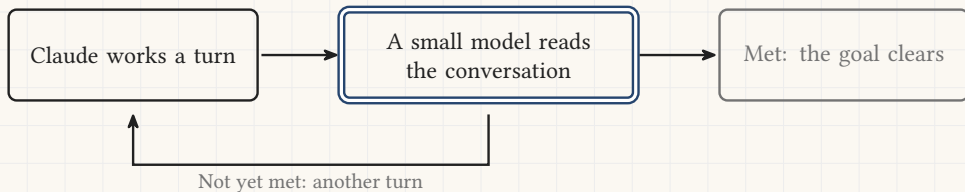
/goal: a small model judges the condition



ooo

```
> /goal all tests in test/auth pass and the lint step is  
clean, or stop after 20 turns
```

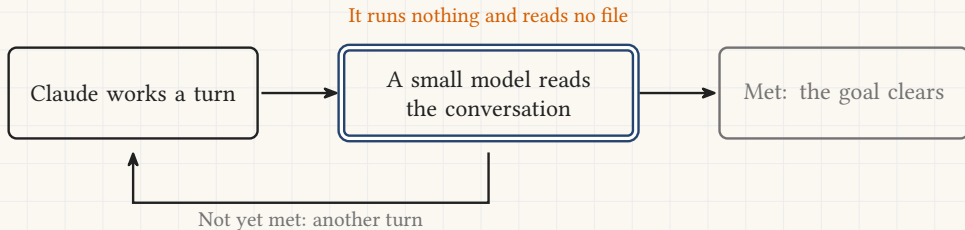
/goal: a small model judges the condition



ooo

```
> /goal all tests in test/auth pass and the lint step is  
clean, or stop after 20 turns
```

/goal: a small model judges the condition



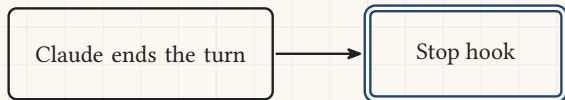
ooo

```
> /goal all tests in test/auth pass and the lint step is  
clean, or stop after 20 turns
```

A Stop hook is a script

Claude ends the turn

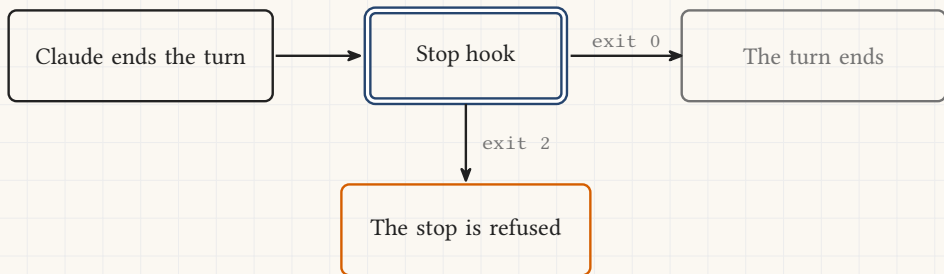
A Stop hook is a script



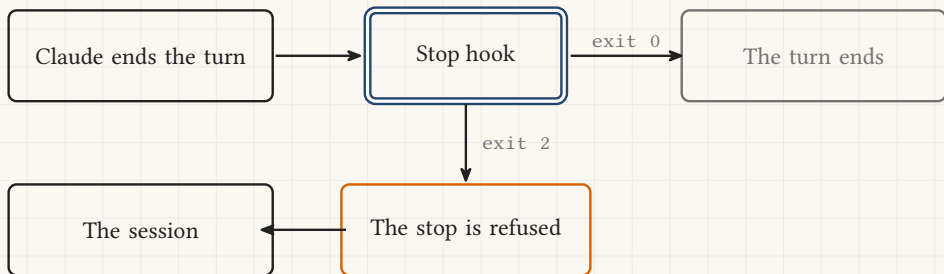
A Stop hook is a script



A Stop hook is a script

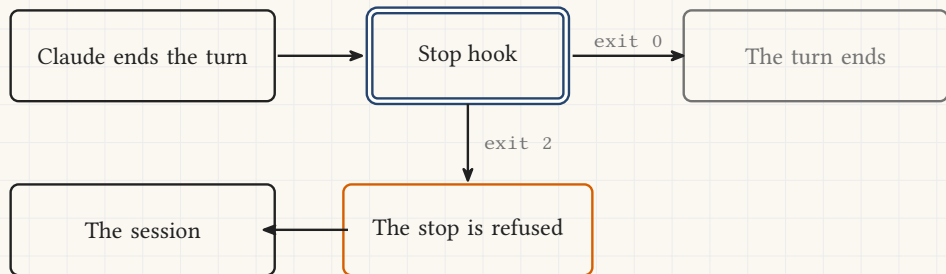


A Stop hook is a script



The script's `stderr`, as the reason

A Stop hook is a script

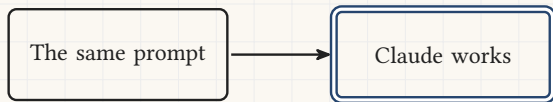


The script's `stderr`, as the reason

<https://code.claude.com/docs/en/hooks>

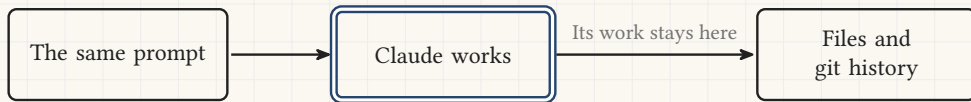
The Ralph loop feeds the same prompt back

Geoffrey Huntley, 2025: <https://ghuntley.com/ralph/>



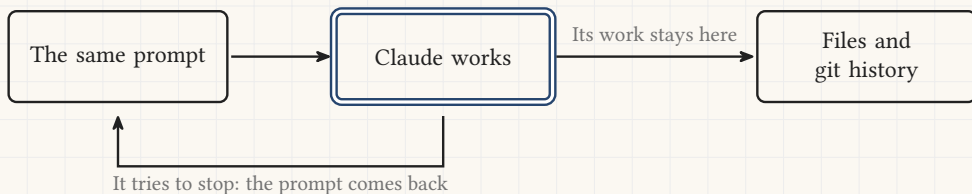
The Ralph loop feeds the same prompt back

Geoffrey Huntley, 2025: <https://ghuntley.com/ralph/>



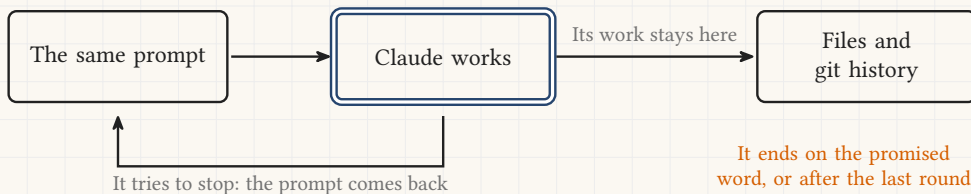
The Ralph loop feeds the same prompt back

Geoffrey Huntley, 2025: <https://ghuntley.com/ralph/>



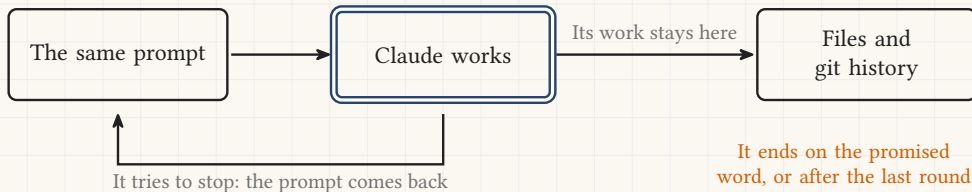
The Ralph loop feeds the same prompt back

Geoffrey Huntley, 2025: <https://ghuntley.com/ralph/>



The Ralph loop feeds the same prompt back

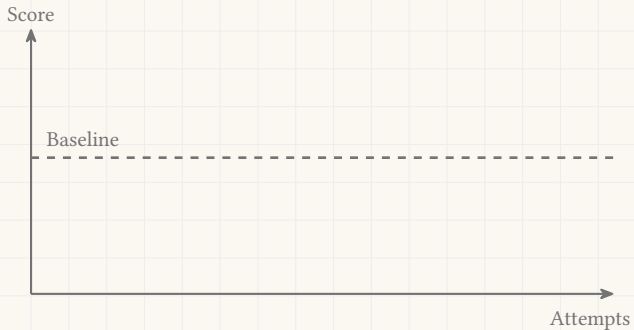
Geoffrey Huntley, 2025: <https://ghuntley.com/ralph/>



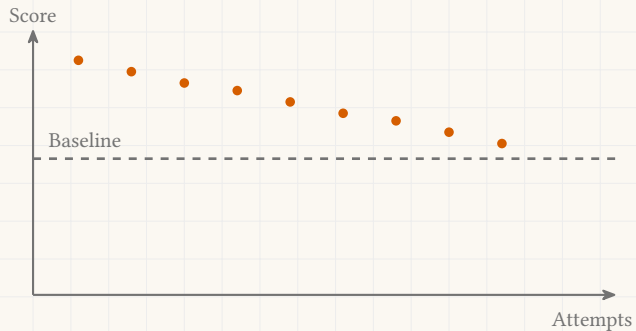
ooo

```
> /ralph-loop "Work through PROMPT.md" --max-iterations 40  
--completion-promise "DONE"
```

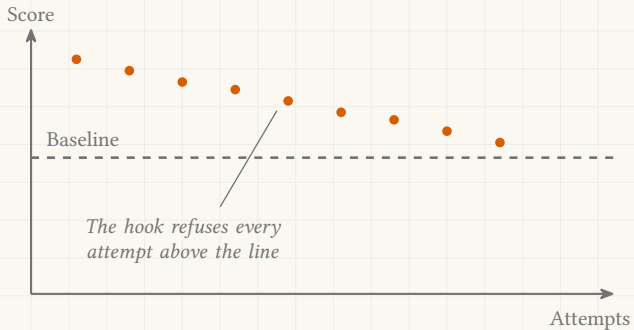
Beat a baseline, or keep going



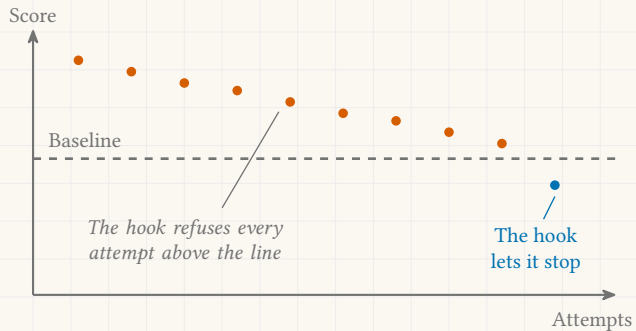
Beat a baseline, or keep going



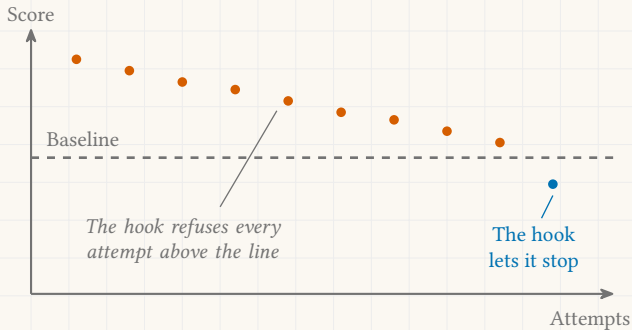
Beat a baseline, or keep going



Beat a baseline, or keep going



Beat a baseline, or keep going

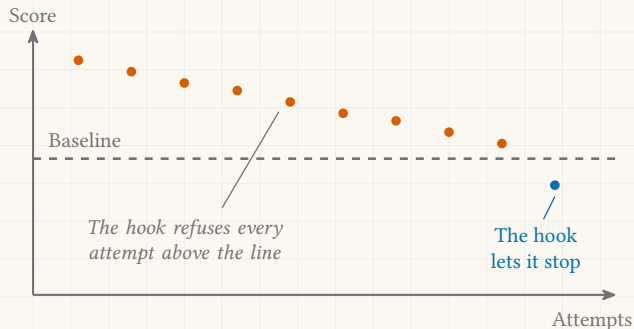


Karpathy's autoresearch

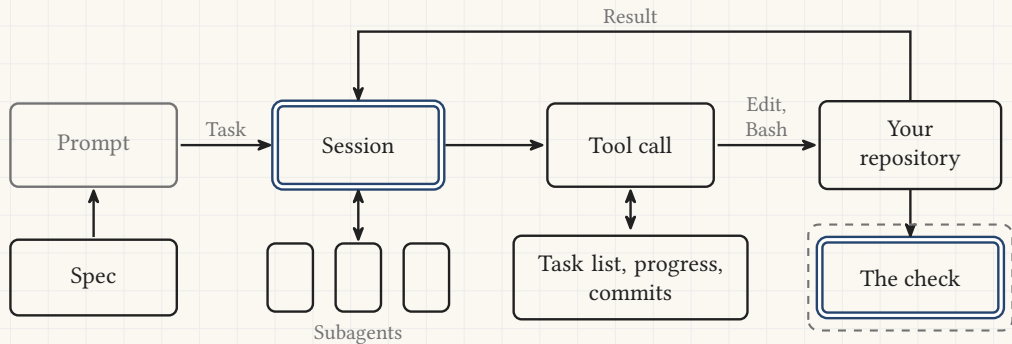
Edits one training file

Trains for exactly
5 minutes on one GPU

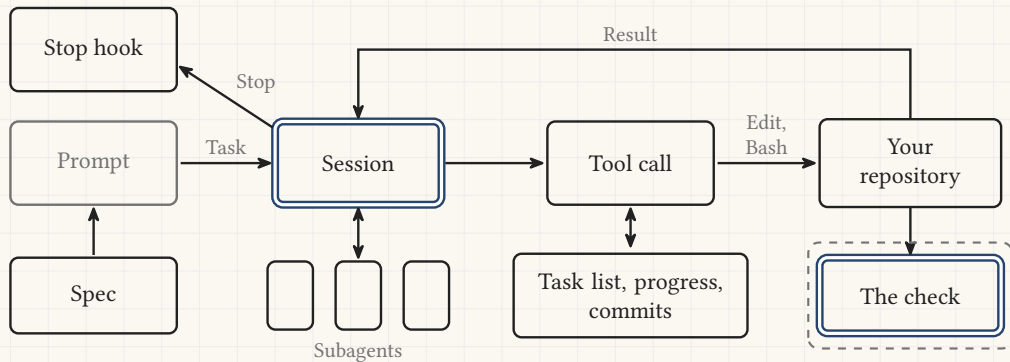
Beat a baseline, or keep going



The loop now stops on a script



The loop now stops on a script



5

Section 5

Worktrees and best-of-N

One session is not enough

The context fills up quickly

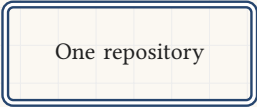
It may change the tests to pass them

It may stop before the task is done

One session is not enough

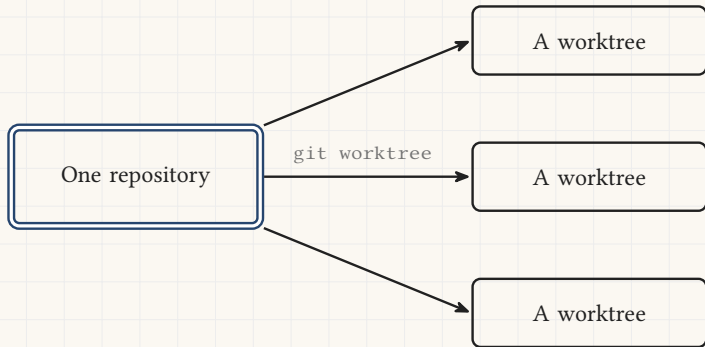
Worktrees and best-of-N

One worktree per session

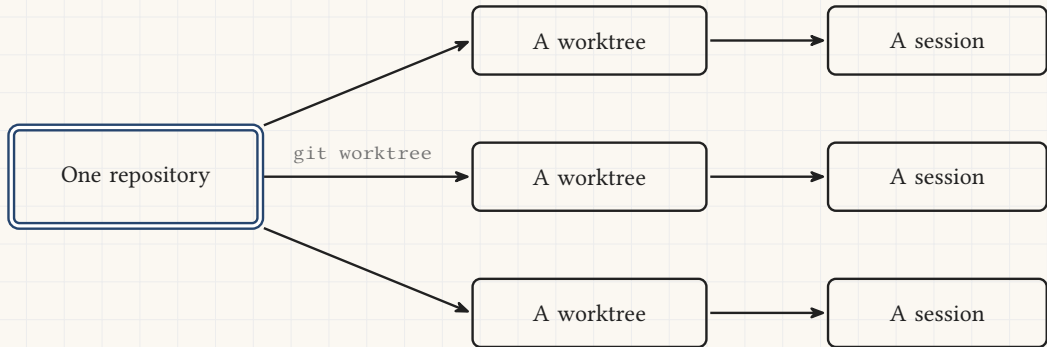


One repository

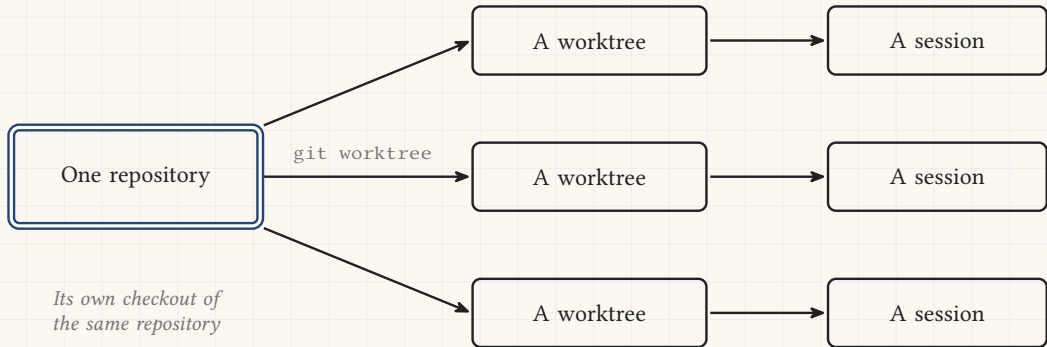
One worktree per session



One worktree per session



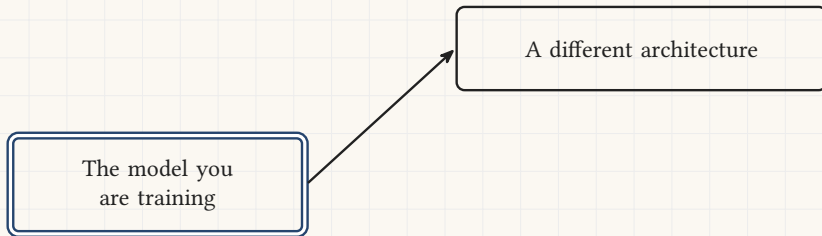
One worktree per session



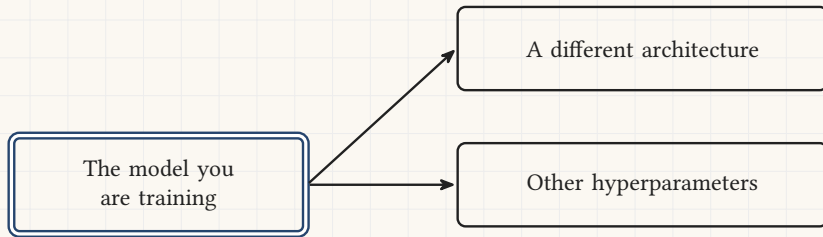
Nobody knows which idea wins

The model you
are training

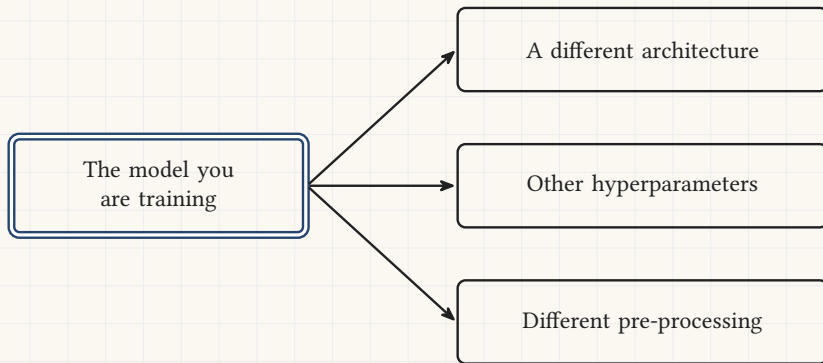
Nobody knows which idea wins



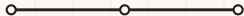
Nobody knows which idea wins



Nobody knows which idea wins

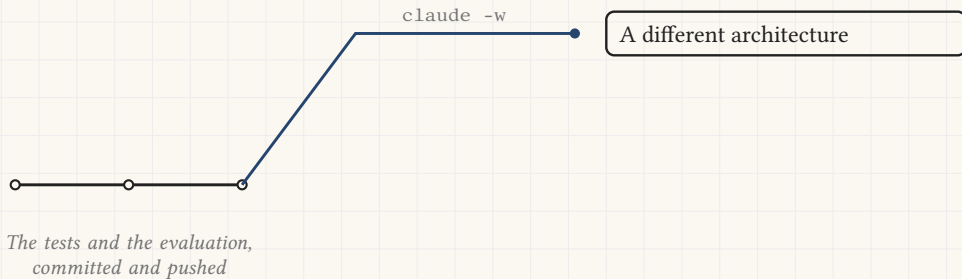


Three attempts, one idea each

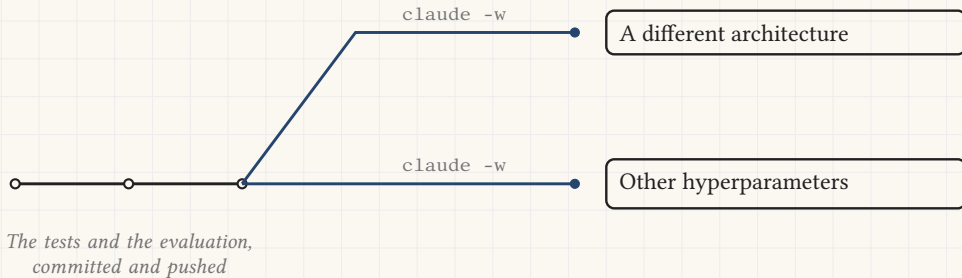


*The tests and the evaluation,
committed and pushed*

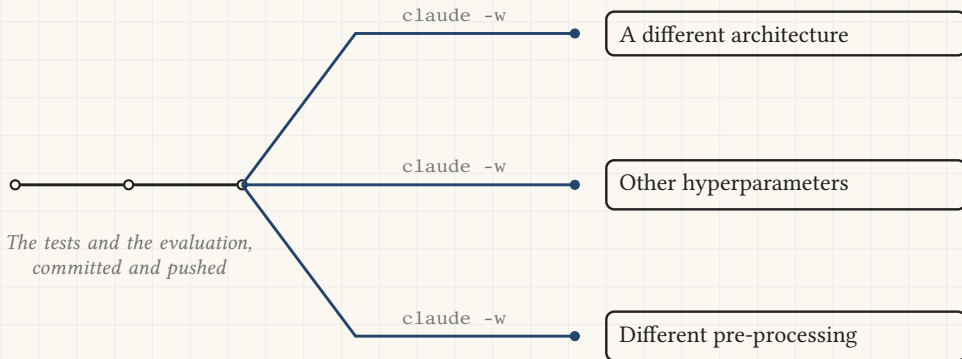
Three attempts, one idea each



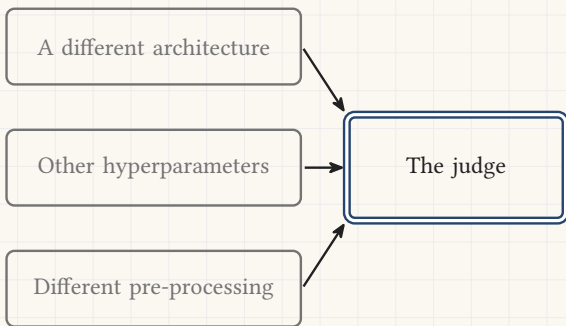
Three attempts, one idea each



Three attempts, one idea each

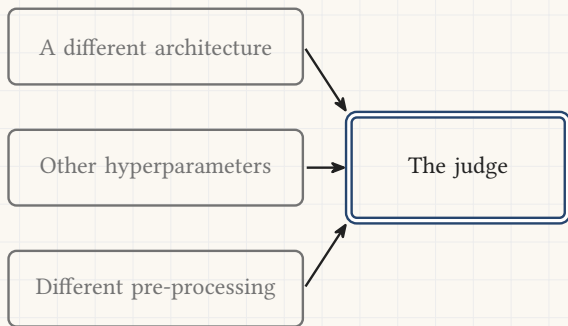


A judge that people own



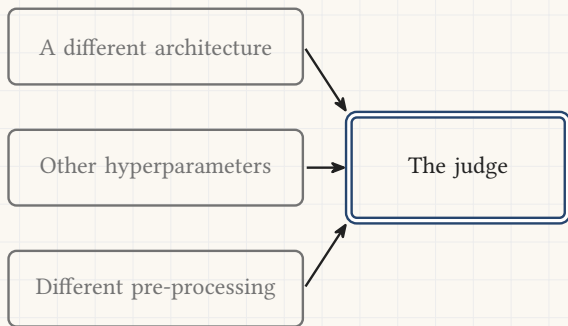
A judge that people own

- ① Changed the tests or the evaluation: disqualified



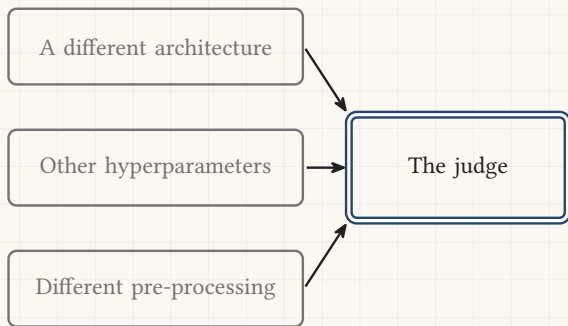
A judge that people own

- ① Changed the tests or the evaluation: disqualified
- ② One attempt at a time, on the same budget



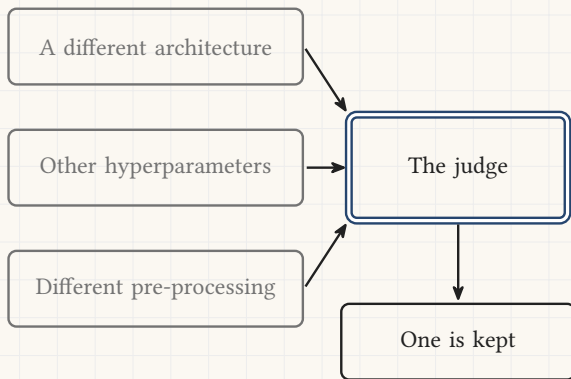
A judge that people own

- ① Changed the tests or the evaluation: disqualified
- ② One attempt at a time, on the same budget
- ③ Measured on data no attempt has seen



A judge that people own

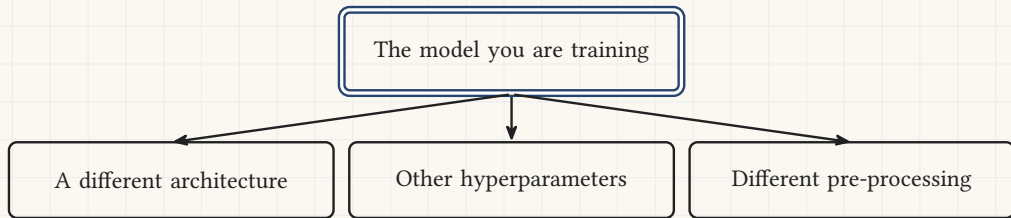
- ① Changed the tests or the evaluation: disqualified
- ② One attempt at a time, on the same budget
- ③ Measured on data no attempt has seen



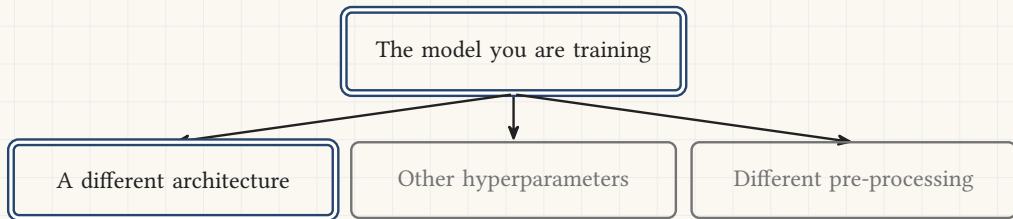
The most promising branch grows again

The model you are training

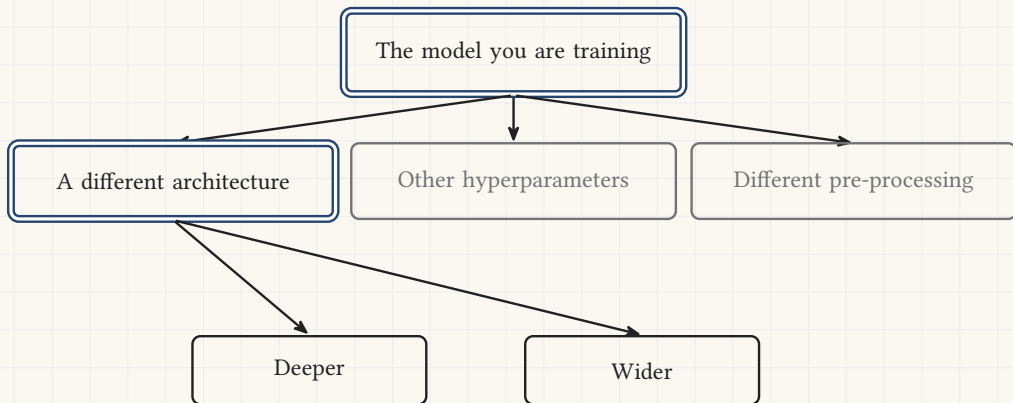
The most promising branch grows again



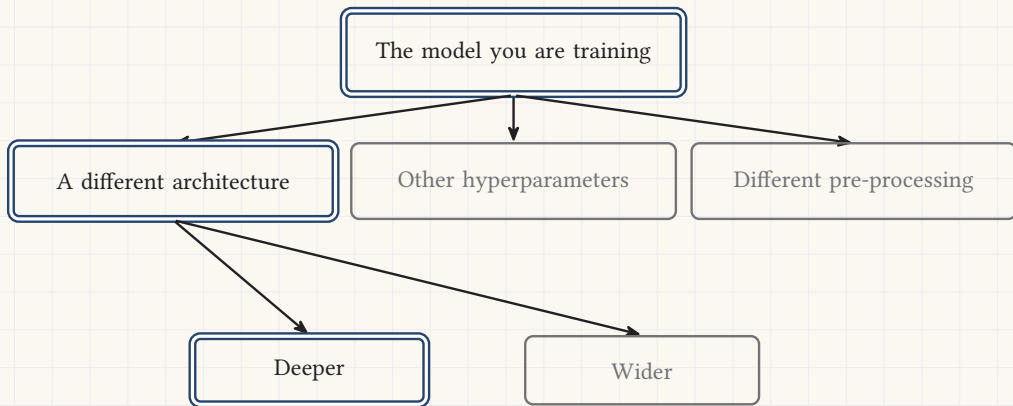
The most promising branch grows again



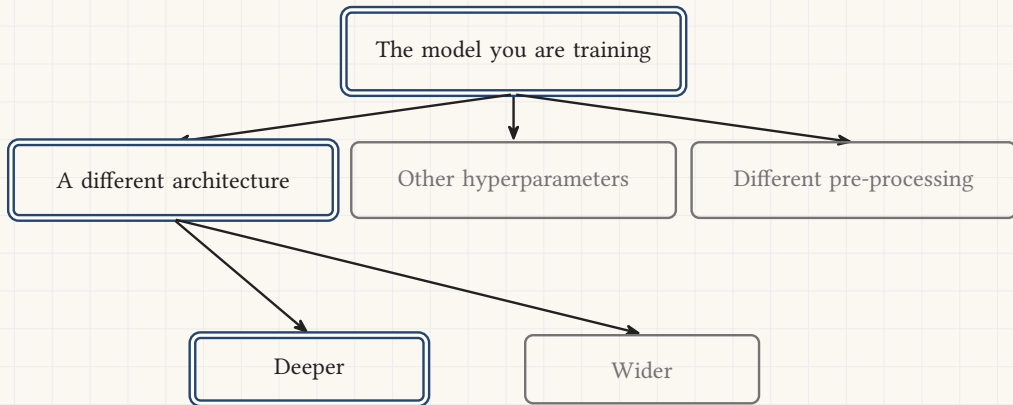
The most promising branch grows again



The most promising branch grows again



The most promising branch grows again



The rest stay as they are

The same three steps, every round

- ① Branch attempts from the current best

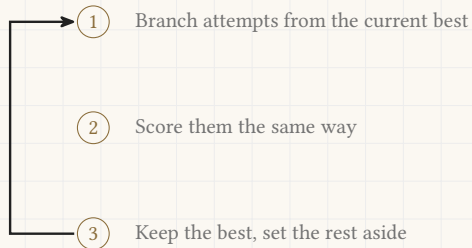
The same three steps, every round

- ① Branch attempts from the current best
- ② Score them the same way

The same three steps, every round

- ① Branch attempts from the current best
- ② Score them the same way
- ③ Keep the best, set the rest aside

The same three steps, every round



Branch the next round from what it kept

6

Section 6

Independent review

One session is not enough

The context fills up quickly

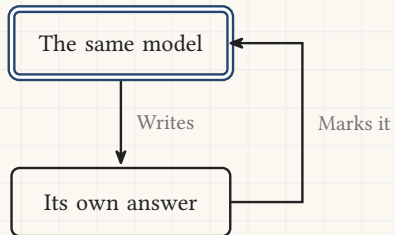
It may change the tests to pass them

It may stop before the task is done

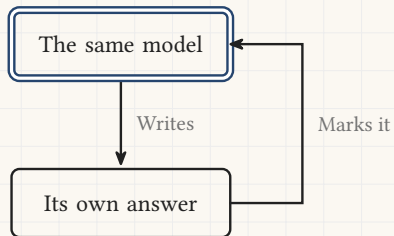
One session is not enough

Independent review

Why a stern prompt does not work



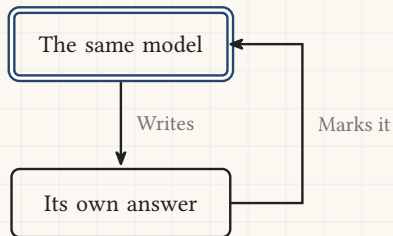
Why a stern prompt does not work



Self-correction without feedback can get worse

Huang et al., ICLR 2024,
arxiv.org/abs/2310.01798

Why a stern prompt does not work



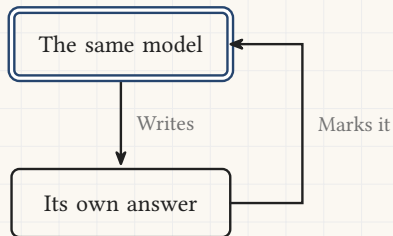
Self-correction without feedback can get worse

Huang et al., ICLR 2024,
arxiv.org/abs/2310.01798

A judge favours its own text

Panickssery, Bowman and Feng, NeurIPS 2024,
arxiv.org/abs/2404.13076

Why a stern prompt does not work



Self-correction without feedback can get worse

Huang et al., ICLR 2024,
arxiv.org/abs/2310.01798

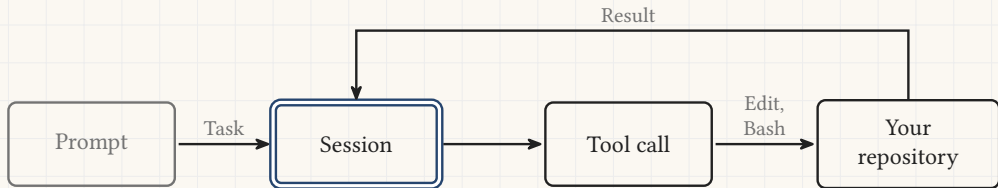
A judge favours its own text

Panickssery, Bowman and Feng, NeurIPS 2024,
arxiv.org/abs/2404.13076

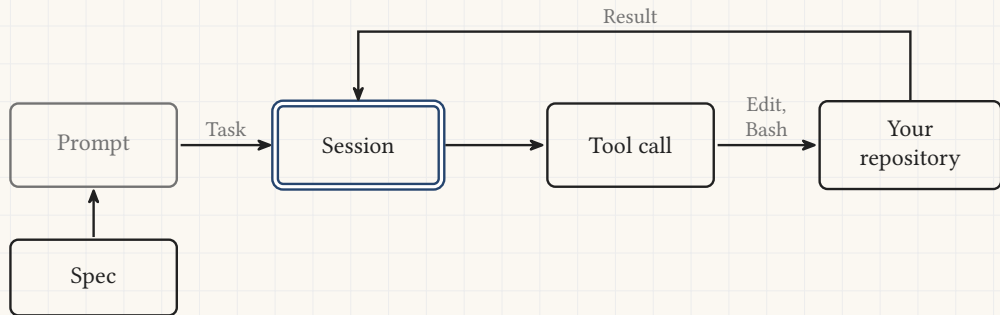
Asked for gaps, a reviewer finds some

Claude Code's own guide, even on sound work

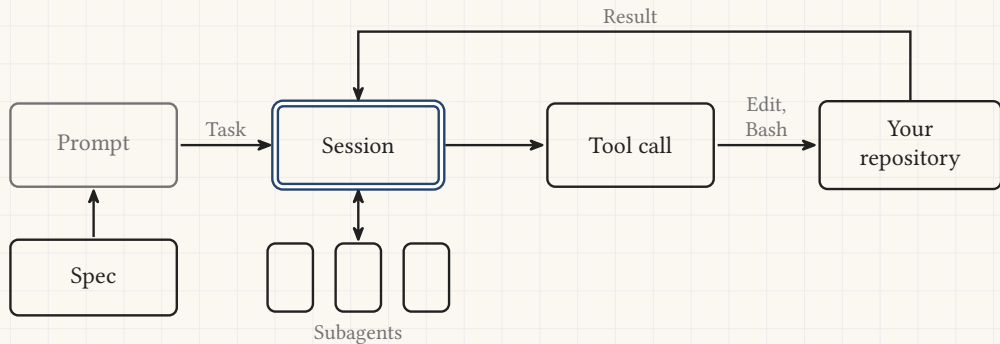
The loop, assembled



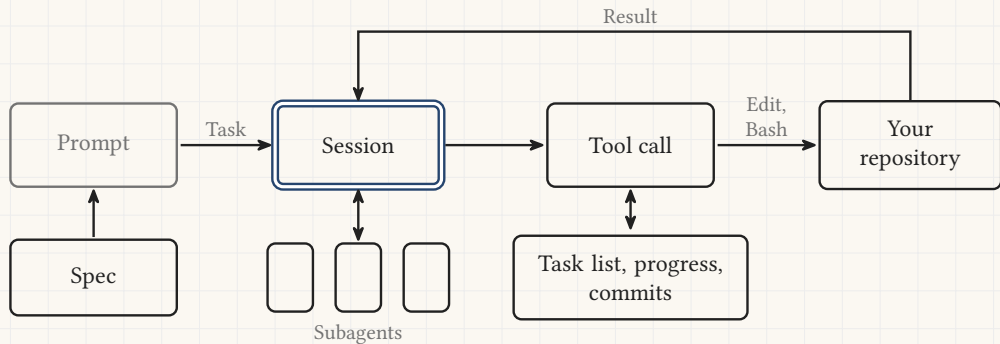
The loop, assembled



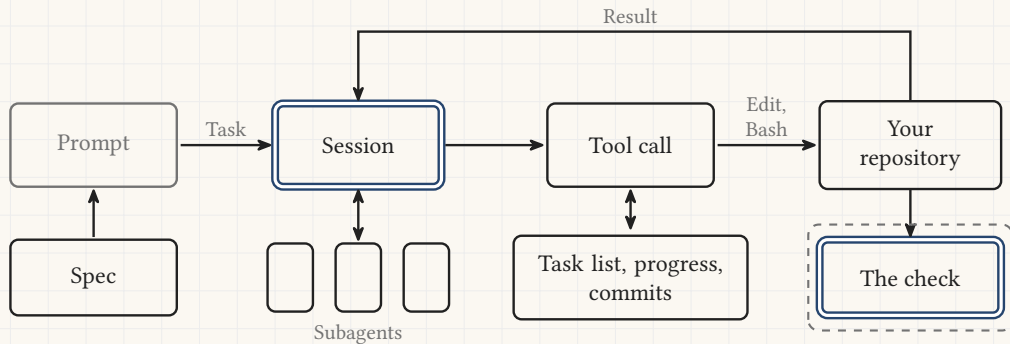
The loop, assembled



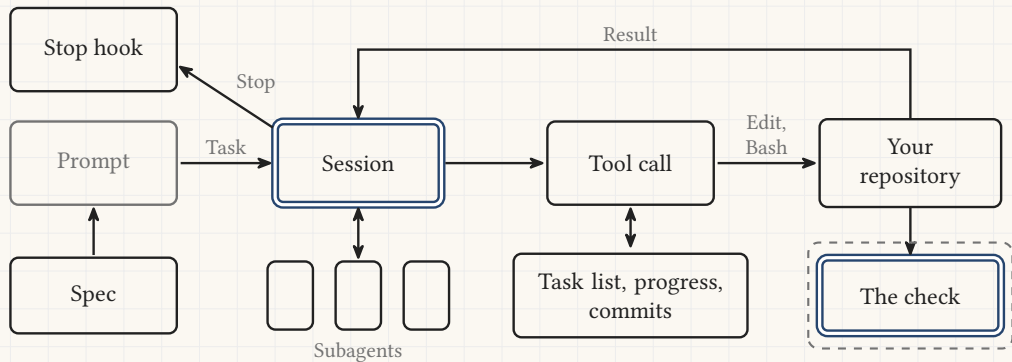
The loop, assembled



The loop, assembled



The loop, assembled



The loop, assembled

